

Introduction

The system is implemented to count words in a distributed environment by groups of words beginning with the same letter. Using Paxos inspired consensus model with these nodes.

Coordinator Node: The coordinator node discovers and broadcasts topology of all proposer, acceptor, and learner nodes in the whole cluster. However, it gives the specific letter ranges to proposers, and reads the input document line by line. It then multicasts each line to all proposers who are supposed to process words within their assigned range.

Proposer Nodes: To process these received amount of text lines and count the words starting with letters from their assigned range, Proposer nodes are responsible. Let each proposer filter words as per its range (A–F) and count and store actual words along with a count. They send the results to two or more acceptor nodes for validation after processing.

Acceptor Nodes: Acceptor nodes validate results sent from proposer nodes in terms of word count to make sure they are correct as well as consistent. The data is accepted by each acceptor, which cross check the data, confirm it follows a required format or logic and forward the validated results to the learner node.

Learner Node: The acceptor nodes finally deliver validated word count results to the learner node which is the final aggregating one. The program combines all accepted data in order to produce the complete summary of counts by the starting letters of words. It iterates finally to produce the output in the required format, that is the letter and count along with a list of words.

Solution Design

1. Letter Range Assignment Algorithm

If all proposers are available, then we want to divide the 26 English alphabet letters (A–Z) among them such that the workload is distributed evenly to all proposer nodes. The purpose of this assignment is to make each proposer responsible for its own subset of letters and process only these words beginning with those letters.

- Total letters = 26 (A to Z)

Let N be the number of proposer nodes.

Calculate the base number of letters each proposer should handle:

- Base range = floor ($26 / N$)

Calculate the number of extra letters that couldn't be evenly distributed:

- Extra letters = $26 \% N$

These extra letters are distributed one by one to the first extra letters proposers, thus some proposers will get one more letter than some other ones.

Sidecar Implementation Details

Sidecar proxy plays an important role in managing inter node communication and logging as it is attached to each node in the distributed system. It handles all HTTP requests and responses between nodes, performing the networking logic as if it were encapsulated inside a single sidecar, from the other side to make main node code cleaner and cleaner. In addition, the sidecar logs all incoming and outgoing messages, including the metadata (timestamps, endpoints, payload summaries, response status). Debugging, auditing and analyzing the system behavior in run time are provided with this logging as they are valuable insight.

Failure Scenarios and Fault Tolerance

1. Proposer Failure:

There is a timeout mechanism used by the coordinator to detect the failure of a proposer node. If the response from a proposer did not come according to the expected time, the coordinator sees that its proposer node is inactive and reassigns its assigned letter range and tasks to another active proposer node. It has ensured that since word counting operations are not interrupted.

2. Acceptor Failure:

Proposers try, to maintain consistency and fault tolerance, to send word count results to at least two acceptor nodes. One acceptor working or not working does not matter, because the majority of the acceptors react. The system therefore continues to reach consensus even with the partial acceptor failures through this approach.

3. Dynamic Node Addition:

The system enables runtime addition of new nodes for dynamic scaling. The coordinator announces itself upon start of a new node. Once the new node is added, the coordinator updates the cluster topology that initial node added into the new node along with broadcasting this configuration to all current nodes to make it use this new node and communicate with others.